

```

{
cout<<"enter a;";
cin>>a;
cout<<"enter b;";
cin>>b;
cout<<"sum= "<<a+b<<endl;
}
sum(int a,int b);
sum(int a, float d,double c);
};
sum :: sum(int x,int y)
{
    a=x;
    b=y;
}
sum :: sum(int p, float q ,double r)
{
    a=p;
    d=q;
    e=r;
}
void main( )
{
clrscr( );
sum 1;
sum m=sum(20,50);
sum n= sum(3,3.2,4.55);
getch( );
}

```

output:
enter a : 3
enter b : 8
sum=11
sum=70
sum=10.75

COPY CONSTRUCTOR:

A copy constructor is used to declare and initialize an object from another object.

Example:-

the statement
integer 12(11);

would define the object 12 and at the same time initialize it to the values of 11.

Another form of this statement is : integer 12=11;

The process of initialization through a copy constructor is known as copy initialization.

Example:-

```

#include<iostream.h>
class code
{
    int id;

```

```

        public
            code ( ) { } //constructor
            code (int a) { id=a; } //constructor
            code(code &x)
            {
                Id=x.id;
            }
            void display( )
            {
                cout<<id;
            }
    };

int main( )
{
    code A(100);
    code B(A);
    code C=A;
    code D;
    D=A;
    cout<<" \n id of A :"; A.display( );
    cout<<" \nid of B :"; B.display( );
    cout<<" \n id of C:"; C.display( );
    cout<<" \n id of D:"; D.display( );
}

```

output :-

```

id of A:100
id of B:100
id of C:100
id of D:100

```

DYNAMIC CONSTRUCTOR:-

The constructors can also be used to allocate memory while creating objects . This will enable the system to allocate the right amount of memory for each object when the objects are not of the same size, thus resulting in the saving of memory.

Allocate of memory to objects at the time of their construction is known as dynamic constructors of objects. The memory is allocated with the help of new operator.

Example:-

```

#include<iostream.h>
#include<string.h>
class string
{
    char *name;

    int length;
public:
    string ( )

```

```

    {
        length=0;
        name= new char [length+1]; /* one extra for \0 */
    }
string( char *s) //constructor 2
{
    length=strlen(s);
    name=new char [length+1];
    strcpy(name,s);
}
void display(void)
{
    cout<<name<<endl;
}
void join(string &a .string &b)
{
    length=a. length +b . length;
    delete name;
    name=new char[length+1]; /* dynamic allocation */
    strcpy(name,a.name);
    strcat(name,b.name);
}
};
int main( )
{
    char * first = “Joseph” ;
    string name1(first),name2(“louis”),name3( “LaGrange”),s1,s2;
    s1.join(name1,name2);
    s2.join(s1,name3);
    name1.display( );
    name2.display( );
    name3.display( );
    s1.display( );
    s2.display( );
}

```

output :-

```

    Joseph
    Louis
    language
    Joseph Louis
    Joseph Louis Language

```

LECTURE-21

DESTRUCTOR:-

A destructor, as the name implies, is used to destroy the objects that have been created by a constructor. Like a constructor, the destructor is a member function whose name is the same as the class name but is preceded by a tilde.

For Example:-

```
~ integer( ) { }
```

A destructor never takes any argument nor does it return any value. It will be invoked implicitly by the compiler upon exit from the program to clean up storage that is no longer accessible. It is a good practice to declare a destructor in a program since it releases memory space for future use.

Delete is used to free memory which is created by new.

Example:-

```
matrix : : ~ matrix( )
{
    for(int i=0; i<11;i++)
        delete p[i];
    delete p;
}
```

IMPLEMENTED OF DESTRUCTORS:-

```
#include<iostream.h>
int count=0;
class alpha
{
public:
    alpha( )
    {
        count ++;
        cout<<"\n no of object created : "<<endl;
    }
    ~alpha( )
    {
        cout<<"\n no of object destroyed : "<<endl;
        count--;
    }
};
```

```
int main( )
{
    cout<<"\n\n enter main \n:";
    alpha A1,A2,A3,A4;
    {
        cout<<"\n enter block 1 :\n";
```

```

        alpha A5;
    }
    {
        cout<<" \n \n enter block2 \n";
        alpha A6;
    }
    cout<<"\n re-enter main \n:";
    return(0);
}

```

output:-

```

enter main
no of object created 1
no of object created 2
no of object created 3
no of object created 4
enter block 1
no of object created 5
no of object destroyed 5
enter block 2
no of object created 5
no of object destroyed 5
re-enter main
no of object destroyed 4
no of object created 3
no of object created 2
no of object created 1

```

Example :-

```

#include<iostream.h>
int x=1;
class abc
{
public:
    abc( )
    {
        x--;
        cout<<"construct the no"<<x<<endl;
    }
    ~abc( )
    {
        cout<<"destruct the no:"<<x<<endl;
        x--;
    }
};

int main( )
{
    abc I1,I2,I3,I4;
    cout<<I1<<I2<<I3<<I4<<endl;
    return(0);
}

```

LECTURE-22 & 23

OPERATOR OVERLOADING:-

Operator overloading provides a flexible option for the creation of new definitions for most of the C++ operators. We can overload all the C++ operators except the following:

- Class members access operator (., .*)
- Scope resolution operator (: :)
- Size operator(sizeof)
- Condition operator (? :)

Although the semantics of an operator can be extended, we can't change its syntax, the grammatical rules that govern its use such as the no of operands precedence and associativity. For example the multiplication operator will enjoy higher precedence than the addition operator.

When an operator is overloaded, its original meaning is not lost. For example, the operator +, which has been overloaded to add two vectors, can still be used to add two integers.

DEFINING OPERATOR OVERLOADING:

To define an additional task to an operator, we must specify what it means in relation to the class to which the operator is applied. This is done with the help of a special function called operator function, which describes the task.

Syntax:-

```
return-type class-name :: operator op( arg-list)
{
    function body
}
```

Where return type is the type of value returned by the specified operation and op is the operator being overloaded. The op is preceded by the keyword operator, operator op is the function name.

operator functions must be either member function, or friend function. A basic difference between them is that a friend function will have only one argument for unary operators and two for binary operators, This is because the object used to invoke the member function is passed implicitly and therefore is available for the member functions. Arguments may be either by value or by reference.

operator functions are declared in the class using prototypes as follows:-

```
vector operator + (vector); // vector addition
vector operator -( ); //unary minus
friend vector operator + (vector, vector); // vector add
friend vector operator -(vector); // unary minus
vector operator - ( vector &a); // subtraction
int operator ==(vector); //comparison
friend int operator ==(vector, vector); // comparison
```

vector is a data type of class and may represent both magnitude and direction or a series of points called elements.

The process of overloading involves the following steps:-

1. Create a class that defines the data type that is used in the overloading operation.
2. Declare the operator function operator op() in the public part of the class
3. It may be either a member function or friend function.
4. Define the operator function to implement the required operations.

Overloaded operator functions can be invoked by expressions such as

op x or x op;

for unary operators and

x op y

for binary operators.

operator op(x);

for unary operator using friend function

operator op(x,y);

for binary operator using friend function.

Unary – operator overloading(using member function):

```
class abc
{
int m,n;
public:
abc()
{
m=8;
n=9;
}
void show()
{
cout<<m<<n;
}
operator -- ()
{
--m;
--n;
}
};
void main()
{
abc x;
x.show();
--x;
```

```
x.show();  
}
```

Unary – - operator overloading(using friend function):

```
class abc  
{  
int m,n;  
public:  
abc()  
{  
m=8;  
n=9;  
}  
void show()  
{  
cout<<m<<n;  
}  
friend operator --(abc &p);  
};  
operator -- (abc &p)  
{  
--p.m;  
--p.n;  
}  
};  
void main()  
{  
abc x;  
x.show();  
operator--(x);  
x.show();  
}
```


Unary operator+ for adding two complex numbers (using member function)

```
class complex
{
float real,img;
public:
complex()
{
    real=0;
    img=0;
}
complex(float r,float i)
{
real=r;
img=i;
}
void show()
{
cout<<real<<"+"i"<<img;
}
complex operator+(complex &p)
{
    complex w;
    w.real=real+p.real;
    w.img=img+p.img;
    return w;
}
};
void main()
{
complex s(3,4);
complex t(4,5);
complex m;
m=s+t;
s.show();
t.show();
m.show();
}
```

Unary operator+ for adding two complex numbers (using friend function)

```
class complex
{
float real,img;
public:
complex()
{
    real=0;
    img=0;
}
complex(float r,float i)
{
real=r;
img=i;
```

```

}
void show()
{
cout<<real<<"+"i"<<img;
}
friend complex operator+(complex &p,complex &q);
};
complex operator+(complex &p,complex &q)
{
    complex w;
    w.real=p.real+q.real;
    w.img=p.img+q.img;
    return w;
}
};
void main()
{
complex s(3,4);complex t(4,5);
complex m;
m=operator+(s,t);
s.show();t.show();
m.show();
}

```

Overloading an operator does not change its basic meaning. For example assume the + operator can be overloaded to subtract two objects. But the code becomes unreachable.

```

class integer
{
    int x, y;
    public:
    int operator + ( ) ;
}
int integer: : operator + ( )
{
    return (x-y) ;
}

```

Unary operators, overloaded by means of a member function, take no explicit argument and return no explicit values. But, those overloaded by means of a friend function take one reference argument (the object of the relevant class).

Binary operators overloaded through a member function take one explicit argument and those which are overloaded through a friend function take two explicit arguments.

Table 7.2

Operator to Overload	Arguments passed to the Member Function	Arguments passed to the Friend Function
Unary Operator	No	1
Binary Operator	1	2

LECTURE-24

Type Conversions

In a mixed expression constants and variables are of different data types. The assignment operations causes automatic type conversion between the operand as per certain rules.

The type of data to the right of an assignment operator is automatically converted to the data type of variable on the left.

Consider the following example:

```
int x;  
float y = 20.123;  
x=y ;
```

This converts float variable y to an integer before its value assigned to x. The type conversion is automatic as far as data types involved are built in types. We can also use the assignment operator in case of objects to copy values of all data members of right hand object to the object on left hand. The objects in this case are of same data type. But of objects are of different data types we must apply conversion rules for assignment.

There are three types of situations that arise where data conversion are between incompatible types.

1. Conversion from built in type to class type.
2. Conversion from class type to built in type.
3. Conversion from one class type to another.

Basic to Class Type

A constructor was used to build a matrix object from an int type array. Similarly, we used another constructor to build a string type object from a char* type variable. In these examples constructors performed a defacto type conversion from the argument's type to the constructor's class type

Consider the following constructor:

```
string :: string (char*a)  
{  
    length = strlen (a);  
    name=new char[len+1];  
    strcpy (name,a);  
}
```

This constructor builds a string type object from a char* type variable a. The variables length and name are data members of the class string. Once you define the constructor in the class string, it can be used for conversion from char* type to string type.

Example

```
string s1 , s2;  
char* name1 = "Good Morning";  
char* name2 = "STUDENTS";  
s1 = string(name1);  
s2 = name2;
```

The program statement

```
si = string (name1);
```

first converts name 1 from char* type to string type and then assigns the string type values to the object s1. The statement

```
s2 = name2;
```

performs the same job by invoking the constructor implicitly.

Consider the following example

```
class time
{
    int hours;
    int minutes;
    public:
    time (int t) // constructor
    {
        hours = t / 60;           //t is inputted in minutes
        minutes = t % 60;
    }
};
```

In the following conversion statements :

```
time T1;           //object T1 created
int period = 160;
T1 = period;       //int to class type
```

The object T1 is created. The variable period of data type integer is converted into class type time by invoking the constructor. After this conversion, the data member hours of T1 will have value 2 and minutes will have a value of 40 denoting 2 hours and 40 minutes.

Note that the constructors used for the type conversion take a single argument whose type is to be converted.

In both the examples, the left-hand operand of = operator is always a class object. Hence, we can also accomplish this conversion using an overloaded = operator.

LECTURE-25

Class to Basic Type

The constructor functions do not support conversion from a class to basic type. C++ allows us to define a overloaded casting operator that convert a class type data to basic type. The general form of an overloaded casting operator function, also referred to as a conversion function, is:

```
operator typename ( )
{
    //Program statmerit .
}
```

This function converts a class type data to typename. For example, the operator double() converts a class object to type double, in the following conversion function:

```
vector:: operator double ( )
{
    double sum = 0 ;
    for(int I = 0; ioize;
    sum = sum + v[i] * v[i ] ; //scalar magnitude
    return sqrt(sum);
}
```

The casting operator should satisfy the following conditions.

- It must be a class member.
- It must not specify a return type.
- It must not have any arguments. Since it is a member function, it is invoked by the object and therefore, the values used for, Conversion inside the function belongs to the object that invoked the function. As a result function does not need an argument.

In the string example discussed earlier, we can convert the object string to char* as follows:

```
string:: operator char*( )
{
    return (str) ;
}
```

One Class to Another Class Type

We have just seen data conversion techniques from a basic to class type and a class to basic type. But sometimes we would like to convert one class data type to another class type.

Example

Obj1 = Obj2 ; //Obj1 and Obj2 are objects of different classes.

Obj1 is an object of class one and Obj2 is an object of class two. The class two type data is converted to class one type data and the converted value is assigned to the Obj1. Since the conversion takes place from class two to class one, two is known as the source and one is known as the destination class.

Such conversion between objects of different classes can be carried out by either a constructor or a conversion function. Which form to use, depends upon where we want the type-conversion function to be located, whether in the source class or in the destination class.

We studied that the casting operator function

```
Operator typename( )
```

Converts the class object of which it is a member to typename. The type name may be a built-in type or a user defined one(another class type) . In the case of conversions between objects,

typename refers to the destination class. Therefore, when a class needs to be converted, a casting operator function can be used. The conversion takes place in the source class and the result is given to the destination class object.

Let us consider a single-argument constructor function which serves as an instruction for converting the argument's type to the class type of which it is a member. The argument belongs to the source class and is passed to the destination class for conversion. Therefore the conversion constructor must be placed in the destination class.

Table 7.3

Conversion	Conversion takes place in	
	Source class	Destination class
Basic to class	Not applicable	Constructor
Class to Basic	Casting operator	Not applicable
Class to class	Casting operator	Constructor

When a conversion using a constructor is performed in the destination class, we must be able to access the data members of the object sent (by the source class) as an argument. Since data members of the source class are private, we must use special access functions in the source class to facilitate its data flow to the destination class.

Consider the following example of an inventory of products in a store. One way of keeping record of the details of the products is to record their code number, total items in the stock and the cost of each item. Alternatively we could just specify the item code and the value of the item in the stock. The following program uses classes and shows how to convert data of one type to another.

```
#include<iostream.h>
#include<conio.h>
class stock2;
class stock1
{
int code, item;
float price;
public:
stock1 (int a, int b, float c)
{
code=a;
item=b;
price=c;
}
void disp( )
{
cout<<"code"<<code <<"\n";
cout<<"Items"<<item <<"\n";
cout<<"Price per item Rs . "<<price <<"\n";
}
int getcode( )
{return code; }
int getitem( )
{return item; }
int getprice( )
{return price;}
```

```

operator float( )
{
return ( item*price );
}
};

class stock2
{
int code;
float val;
public:
stock2()
{
code=0; val=0;
}
stock2(int x, float y)
{
code=x; val=y;
}
void disp( )
{
cout<< "code"<<code << "\n";
cout<< "Total Value Rs . " <<val << "\n"
}
stock2 (stock1 p)
{
code=p . getcode ( ) ;
val=p.getitem( ) * p. getprice ( ) ;
}
};

void main ( )
{
Stock1 i1(101, 10,125.0);
Stock2 i2;
float tot_val;
tot_val=i1 ;
i2=i1 ;
cout<<" Stock Details-stock1-type" << "\n";
i1 . disp ( ) ;
cout<<" Stock value"<< "\n";
cout<< tot_val<< "\n";
cout<<" Stock Details-stock2-type"<< " \n";
i2 .disp( ) ;
getch ( ) ;
}

```

You should get the following output.

Stock Details-stock1-type

code 101

Items 10

Price per item Rs. 125

Stock value

1250

Stock Details-stock2-type

code 10 1

Total Value Rs. 1250

LECTURE-26

Inheritance:

Reaccessability is yet another feature of OOP's. C++ strongly supports the concept of reusability. The C++ classes can be used again in several ways. Once a class has been written and tested, it can be adopted by another programmers. This is basically created by defining the new classes, reusing the properties of existing ones. The mechanism of deriving a new class from an old one is called 'INHERTTENCE'. This is often referred to as IS-A' relationship because very object of the class being defined "is" also an object of inherited class. The old class is called 'BASE' class and the new one is called'DERIEVED'class.

Defining Derived Classes

A derived class is specified by defining its relationship with the base class in addition to its own details. The general syntax of defining a derived class is as follows:

```
class d_classname : Access specifier baseclass name
{
    —
    — // members of derived class
};
```

The colon indicates that the a-class name is derived from the base class name. The access specifier or the visibility mode is optional and, if present, may be public, private or protected. By default it is private. Visibility mode describes the status of derived features e.g.

```
class xyz      //base class
{
    members of xyz
};
class ABC : public xyz  //public derivation
{
    members of ABC
};
class ABC : XYZ  //private derivation (by default)
{
    members of ABC
};
```

In the inheritance, some of the base class data elements and member functions are inherited into the derived class. We can add our own data and member functions and thus extend the functionality of the base class. Inheritance, when used to modify and extend the capabilities of the existing classes, becomes a very powerful tool for incremental program development.

Single Inheritance

When a class inherits from a single base class, it is known as single inheritance. Following program shows the single inheritance using public derivation.

```
#include<iostream.h>
#include<conio.h>
class worker
{
```

```

        int age;
        char name [10];
        public:
        void get ( );
};
void worker :: get ( )
{
    cout <<"your name please"
    cin >> name;
    cout <<"your age please" ;
    cin >> age;
}
void worker :: show ( )
{
    cout <<"In My name is :"<<name<<"In My age is :"<<age;
}
class manager :: public worker //derived class (publicly)
{
    int now;
    public:
    void get ( );
    void show ( );
};
void manager :: get ( )
{
    worker :: get ( ); //the calling of base class input fn.
    cout << "number of workers under you";
    cin >> now;
    cin>>name>>age;
} // ( if they were public )
void manager :: show ( )
{
    worker :: show ( ); //calling of base class o/p fn.
    cout <<"in No. of workers under me are: "<< now;
}

main ( )
{
    clrscr ( );
    worker W1;
    manager M1;
    M1.get ( );
    M1.show ( );
}

```

If you input the following to this program:

Your name please

Ravinder

Your age please

27

number of workers under you

30

Then the output will be as follows:

My name is : Ravinder

My age is : 27

No. of workers under me are : 30

The following program shows the single inheritance by private derivation.

```
#include<iostream.h>
#include<conio.h>
class worker //Base class declaration
{
    int age;
    char name [10] ;
    public:
    void get ( ) ;
    void show ( ) ;
};
void worker : : get ( )
{
    cout << "your name please" ;
    cin >> name;
    cout << "your age please";
    cin >>age;
}
void worker : show ( )
{
    cout << "in my name is: " <<name<< "in" << "my age is : " <<age;
}
class manager : worker //Derived class (privately by default)
{
    int now;
    public:
    void get ( ) ;
    void show ( ) ;
};
void manager : : get ( )
{
    worker : : get ( ) ; //calling the get function of base
    cout << "number of worker under you"; class which is
    cin >> now;
}
void manager : : show ( )
{
    worker : : show ( ) ;
    cout << "in no. of worker under me are : " <<now;
}
main ( )
{
```

```

        clrscr ( ) ;
        worker wl ;
        manager ml;
        ml.get ( ) ;
        ml.show ( );
    }

```

The following program shows the single inheritance using protected derivation

```

#include<conio.h>
#include<iostream.h>
class worker          //Base class declaration
{ protected:
    int age; char name [20];
    public:
    void get ( );
    void show ( );
};
void worker :: get ( )
{
    cout >> "your name please";
    cin >> name;
    cout << "your age please";
    cin >> age;
}
void worker :: show ( )
{
    cout << "in my name is: " << name << "in my age is " << age;
}
class manager:: protected worker  // protected inheritance
{
    int now;
    public:
    void get ( );
    void show ( ) ;
};
void manager : : get ( )
{
    cout << "please enter the name In";
    cin >> name;
    cout<< "please enter the age In"; //Directly inputting the data
    cin >> age;    members of base class
    cout << " please enter the no. of workers under you:";
    cin >> now;
}
void manager : : show ( )

{
    cout << "your name is : "<<name<<" and age is : "<<age;
    cout <<"In no. of workers under your are : "<<now;
main ( )
{
    clrscr ( ) ;
    manager ml;
    ml.get ( ) ;

```

```

        cout << "\n\n";
        ml.show ( );
    }

```

Making a Private Member Inheritable

Basically we have visibility modes to specify that in which mode you are deriving the another class from the already existing base class. They are:

- Private:** when a base class is privately inherited by a derived class, 'public members' of the base class become private members of the derived class and therefore the public members of the base class can be accessed by its own objects using the dot operator. The result is that we have no member of base class that is accessible to the objects of the derived class.
- Public:** On the other hand, when the base class is publicly inherited, 'public members' of the base class become 'public members' of derived class and therefore they are accessible to the objects of the derived class.
- Protected:** C++ provides a third visibility modifier, protected, which serve a little purpose in the inheritance. A member declared as protected is accessible by the member functions within its class and any class immediately derived from it. It cannot be accessed by functions outside these two classes.

The below mentioned table summarizes how the visibility of members undergo modifications when they are inherited

Base Class Visibility	Derived Class Visibility		
	Public	Private	Protected
Private	X	X	X
Public	Public	Private	Protected
Protected	Protected	Private	Protected

The private and protected members of a class can be accessed by:

- A function i.e. friend of a class.
- A member function of a class that is the friend of the class.
- A member function of a derived class.

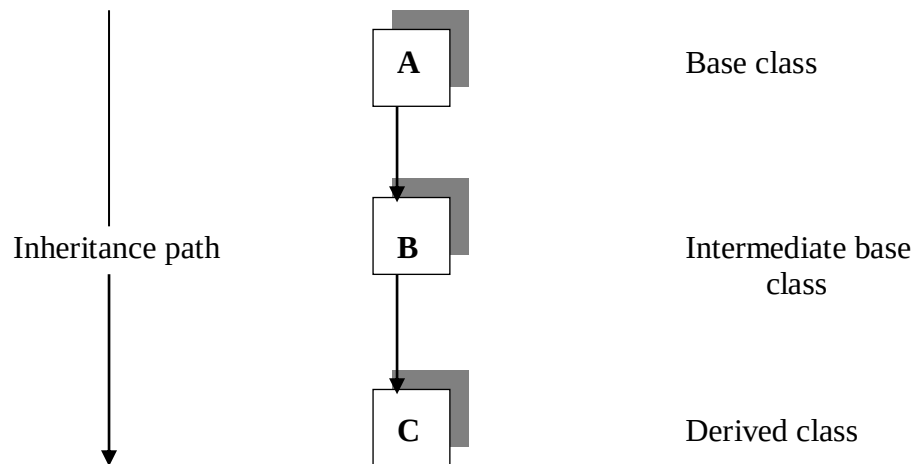
Student Activity

- Define Inheritance. What is the inheritance mechanism in C++?
- What are the advantage of Inheritance?
- What should be the structure of a class when it has to be a base for other classes?

LECTURE-27

Multilevel Inheritance

When the inheritance is such that, the class A serves as a base class for a derived class B which in turn serves as a base class for the derived class C. This type of inheritance is called 'MULTILEVEL INHERITENCE'. The class B is known as the 'INTERMEDIATE BASE CLASS' since it provides a link for the inheritance between A and C. The chain ABC is called 'ITNHERITENCE*PATH' for e.g.



The declaration for the same would be:

```
Class A
{
//body
}
Class B : public A
{
//body
}
Class C : public B
{
//body
}
```

This declaration will form the different levels of inheritance.

Following program exhibits the multilevel inheritance.

```
#include<iostream.h>
#include<conio.h>
class worker    // Base class declaration
{
    int age;
    char name [20] ;
    public;
    void get( ) ;
```

```

        void show( ) ;
    }

void worker: get ( )
{
    cout << "your name please" ;
    cin >> name;
    cout << "your age please" ;
}

void worker : : show ( )
{
    cout << "In my name is : " << name << " In my age is : " << age;
}

class manager : public worker //Intermediate base class derived
{
    //publicly from the base class
    int now;
    public:
    void get ( ) ;
    void show( ) ;
};

void manager :: get ( )
{
    worker : : get ( ) ;    //calling get ( ) fn. of base class
    cout << "no. of workers under you:";
    cin >> now;
}

void manager : : show ( )
{
    worker : : show ( ) ;    //calling show ( ) fn. of base class
    cout << "In no. of workers under me are: " << now;
}

class ceo: public manager    //declaration of derived class
{
    //publicly inherited from the
    int nom;    //intermediate base class
    public:
    void get ( ) ;
    void show ( ) ;
};

void ceo : : get ( )
{
    manager : : get ( ) ;
    cout << "no. of managers under you are:"; cin >> nom;
}

void manager : : show ( )
{
    cout << "In the no. of managers under me are: In";
    cout << "nom;
}

```

```
main ( )
{
    clrscr ( );
    ceo cl ;
    cl.get ( ); cout << "\n\n";
    cl.show ( );
}
```

Worker

Private: int age; char name[20];
Protected:
Private: int age; char name[20];

Manager:Worker

Private: int now;
Protected:
Public: void get() void show() worker ::get() worker ::get()

Ceo: Manager

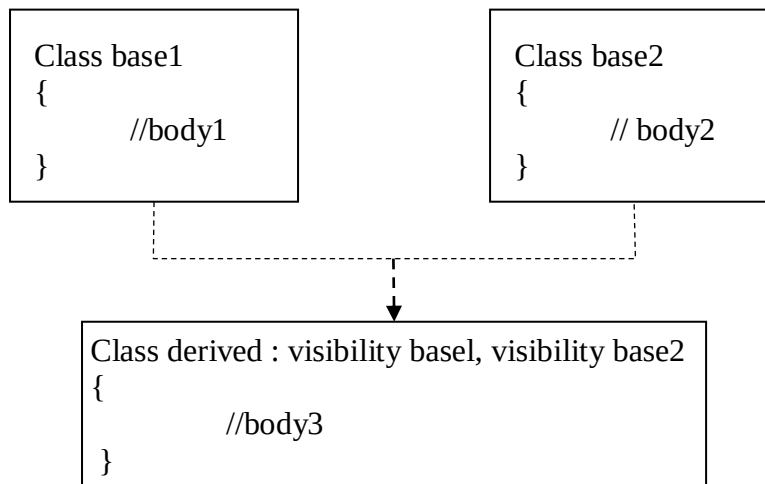
Public:
Protected:
Public:

All the inherited
members

Multiple Inheritances

A class can inherit the attributes of two or more classes. This mechanism is known as ‘MULTIPLE INHERITENCE’. Multiple inheritance allows us to combine the features

of several existing classes as a starting point for defining new classes. It is like the child inheriting the physical feature of one parent and the intelligence of another. The syntax of the derived class is as follows:



Where the visibility refers to the access specifiers i.e. public, private or protected. Following program shows the multiple inheritance.

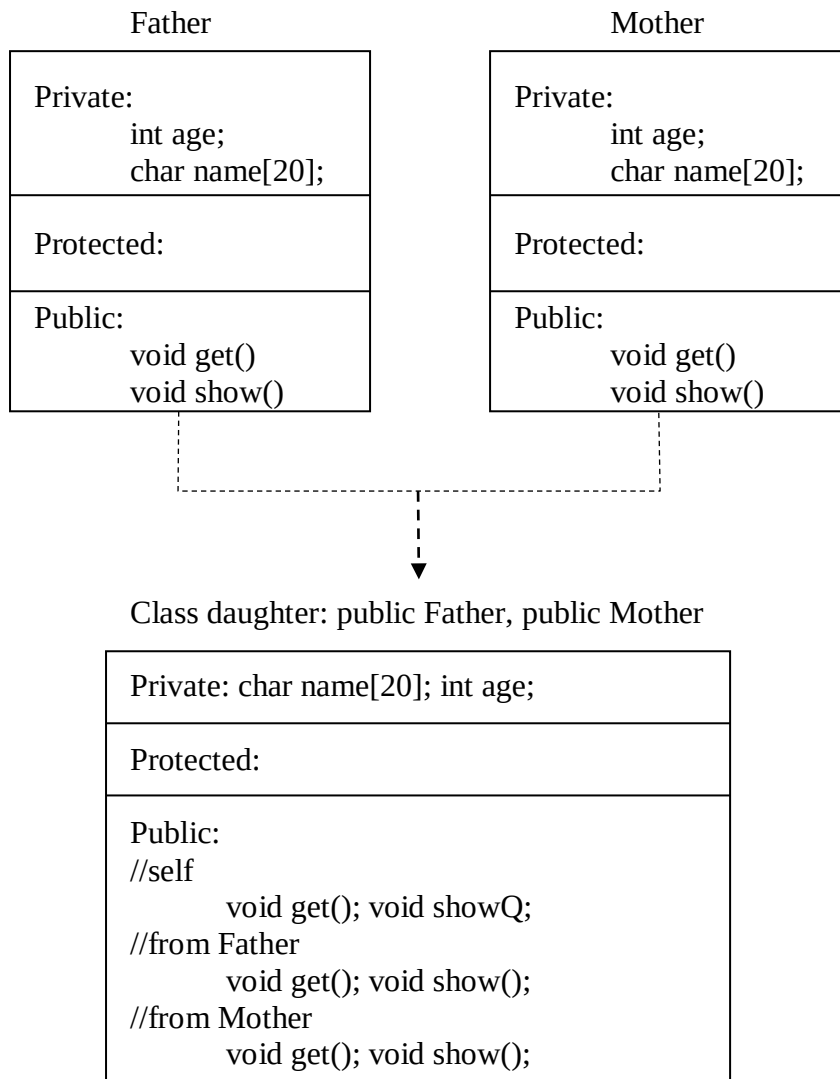
```
#include<iostream.h>
#include<conio . h>
class father          //Declaration of base class1
{
    int age ;
    char flame  [20] ;
    public:
    void get ( ) ;
    void show ( ) ;
};
void father : : get ( )
{
    cout << “your father name please”;
    cin >> name;
    cout << “Enter the age”;
    cin >> age;
}
void father : : show ( )
{
    cout<< “In my father’s name is: ‘ <<name<< “In my father’s age is:<<age;
}
class mother          //Declaration of base class 2
{
    char name [20] ;
    int age ;
```

```

public:
void get ( )
{
    cout << "mother's name please" << "In";
    cin >> name;
    cout << "mother's age please" << "in";
    cin >> age;
}
void show ( )
{
    cout << "In my mother's name is: " << name;
    cout << "In my mother's age is: " << age;
}
class daughter : public father, public mother //derived class inheriting
{
    //publicly
    char name [20] ; //the features of both the base class
    int std;
public:
    void get ( ) ;
    void show ( ) ;
};
void daughter :: get ( )
{
    father :: get ( ) ;
    mother :: get ( ) ;
    cout << "child's name: ";
    cin >> name;
    cout << "child's standard";
    cin >> std;
}
void daughter :: show ( )
{
    father :: show ( ) ;
    mother :: show ( ) ;
    cout << "In child's name is : " << name;
    cout << "In child's standard: " << std;
}
main ( )
{
    clrscr ( ) ;
    daughter d1;
    d1.get ( ) ;
    d1.show ( ) ;
}

```

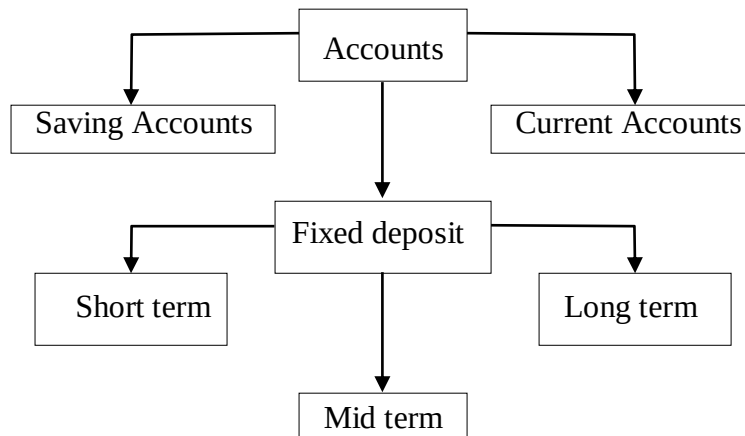
Diagrammatic Representation of Multiple Inheritance is as follows:



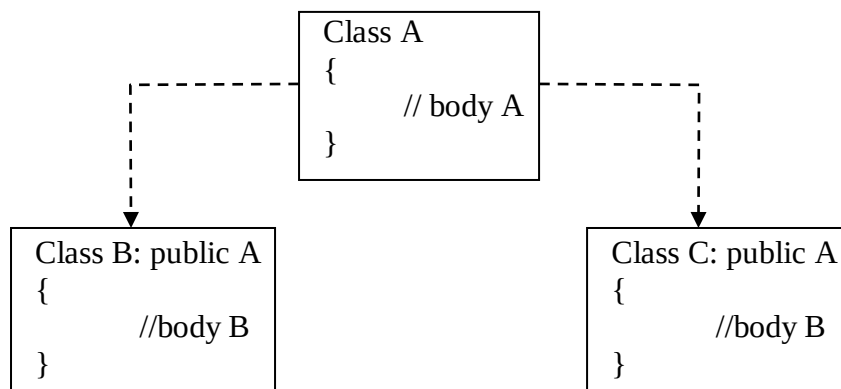
LECTURE-28

Hierarchical Inheritance

Another interesting application of inheritance is to use it as a support to a hierarchical design of a class program. Many programming problems can be cast into a hierarchy where certain features of one level are shared by many others below that level for e.g.



In general the syntax is given as



In C++, such problems can be easily converted into hierarchies. The base class will include all the features that are common to the subclasses. A sub-class can be constructed by inheriting the features of base class and so on.

```
// Program to show the hierarchical inheritance
#include<iostream.h>
#include<conio.h>
class father          //Base class declaration
{
    int age;
    char name [15];
public:
    void get ( )
    {
        cout<< "father name please"; cin >> name;
```

```

        cout<< "father's age please"; cin >> age;
    }
    void show ( )
    {
        cout << "In father's name is ": "<<name;
        cout << "In father's age is: "<< age;
    }
};
class son : public father      //derived class 1
{
    char name [20] ;
    int age ;
    public;
    void get ( ) ;
    void show ( ) ;
};
void son : : get ( )

{
    father :: get ( ) ;
    cout << "your (son) name please" << "in"; cin >>name;
    cout << "your age please" << "In"; cin>>age;
}
void son :: show ( )
{
    father : : show ( ) ;
    cout << "In my name is : " <<name;
    cout << "In my age is : " <<age;
}
class daughter : public father      //derived class 2.
{
    char name [15] ;
    int age;
    public:
    void get ( )
    {
        father : : get ( ) ;
        cout << "your (daughter's) name please In" cin>>name;
        cout << "your age please In"; cin >>age;
    }
    void show ( )
    {
        father : : show ( ) ;
        cout << "in my father name is: " << name << "
        In and his age is : "<<age;
    }
};
main ( )
{
    clrscr ( ) ;

```

```

        son S1;
        daughter D1 ;
        S1. get ( ) ;
        D1. get ( ) ;
        S1 .show( ) ;
        D1. show ( ) ;
    }

```

Hybrid Inheritance

There could be situations where we need to apply two or more types of inheritance to design a program. Basically Hybrid Inheritance is the combination of one or more types of the inheritance. Here is one implementation of hybrid inheritance.

//Program to show the simple hybrid inheritance

```

#include<iostream.h>
#include<conio.h>
class student          //base class declaration
{
    protected:
    int r_no;
    public:
        void get_n (int a)
        {
            r_no =a;
        }
        void put_n (void)
        {
            cout << "Roll No. : "<< r_no;
            cout << "In";
        }
};
class test : public student
{          //Intermediate base class
    protected : int parti, par 2;

    public :
        void get_m (int x, int y) {
            parti = x; part 2 = y; }
        void put_m (void) {
            cout << "marks obtained: " << "In"
                << "Part 1 = " << part1 << "in"
                << "Part 2 = " << part2 << "In";
        }
};
class sports          // base for result
{
    protected : int score;
    public:
        void get_s (int s) {
            score = s }
        void put_s (void) {
            cout << " sports wt. : " << score << "\n\n";

```

```

        }
    };
class result : public test, public sports //Derived from test
        & sports
{
    int total;
public:
    void display (void);
};

void result::display (void)
{
    total = part1 + part2 + score;
    put_n ( );
    put_m ( );
    put_S ( );
    cout << "Total score: " << total << "\n"
}
main ( )
{
    clrscr ( ) ;
    result S1;
    S1.get_n (347) ;
    S1.get_m (30, 35);
    S1.get_s (7) ;
    S1.display ( ) ;
}

```

Student Activity

1. What is the major use of multilevel Inheritance?
2. How are arguments sent to the base constructors in multiple inheritance? Whose responsibility is it.
3. What is the difference between hierarchical and hybrid Inheritance.

LECTURE-29

Virtual Base Classes

We have just discussed a situation which would require the use of both multiple and multi level inheritance. Consider a situation, where all the three kinds of inheritance, namely multi-level, multiple and hierarchical are involved.

Let us say the 'child' has two direct base classes 'parent1' and 'parent2' which themselves has a common base class 'grandparent'. The child inherits the traits of 'grandparent' via two separate paths. It can also be inherit directly as shown by the broken line. The grandparent is sometimes referred to as 'INDIRECT BASE CLASS'. Now, the inheritance by the child might cause some problems. All the public and protected members of 'grandparent' are inherited into 'child' twice, first via 'parent1' and again via 'parent2'. So, there occurs a duplicacy which should be avoided.

The duplication of the inherited members can be avoided by making common base class as the virtual base class: for e.g.

```
class g_parent
{
    //Body
};
class parent1: virtual public g_parent
{
    // Body
};

class parent2: public virtual g_parent
{
    // Body
};

class child : public parent1, public parent2
{
    // body
};
```

When a class is virtual base class, C++ takes necessary care to see that only one copy of that class is inherited, regardless of how many inheritance paths exists between virtual base class and derived class. Note that keywords 'virtual' and 'public' can be used in either order.

//Program to show the virtual base class

```
#include<iostream.h>
#include<conio . h>
class student          // Base class declaration
{
    protected:
    int r_no;
    public:
    void get_n (int a)
    { r_no = a; }
    void put_n (void)
    { cout << "Roll No. " << r_no << "ln"; }
};
```



```

class test : virtual public student // Virtually declared common
{
    //base class 1
    protected:
    int part1;
    int part2;
    public:
    void get_m (int x, int y)
    { part1= x; part2=y;}
    void putm (void)
    {
        cout << "marks obtained: " << "\n";
        cout << "part1 = " << part1 << "\n";
        cout << "part2 = " << part2 << "\n";
    }
};

class sports : public virtual student // virtually declared common
{
    //base class 2
    protected:
    int score;
    public:
    void get_s (int a) {
        score = a ;
    }
    void put_s (void)
    { cout << "sports wt.: " << score << "\n"; }
};

class result: public test, public sports //derived class
{
    private : int total ;
    public:
    void show (void) ;
};

void result : : show (void)
{ total = part1 + part2 + score ;
  put_n ( ) ;
  put_m ( ) ;
  put_s ( ) ; cout << "\n total score= " << total << "\n" ;
}

main ( )
{
    clrscr ( ) ;
    result S1 ;
    S1.get_n (345)
    S1.get_m (30, 35) ;
    S1.get-S (7) ;
    S1. show ( ) ;
}

//Program to show hybrid inheritance using virtual base classes
#include<iostream.h>
#include<conio.h>
Class A
{

```

```

protected:
    int x;
public:
    void get (int) ;
    void show (void) ;
};

void A :: get (int a)
    { x = a ; }
void A :: show (void)
    { cout << X ;}
Class A1 : Virtual Public A
{

```

```

protected:
    int y ;
public:
    void get (int) ;
    void show (void);
};

void A1 :: get (int a)
    { y = a;}
void A1 :: show (void)
{
    cout <<y ;
}
class A2 : Virtual public A
{
protected:
    int z ;
public:
    void get (int a)
        { z =a;}
    void show (void)
        { cout << z;}
};

class A12 : public A1, public A2
{
int r, t ;
public:
    void get (int a)
        { r = a;}
    void show (void)
        { t = x + y + z + r ;
          cout << "result =" << t ;
        }
};

main ( )
{
    clrscr ( ) ;

```

```
A12 r;  
r.A :: get (3);  
r.A1 :: get (4);  
r.A2 :: get (5);  
r.get (6);  
r . show ();  
}
```

LECTURE-30

Polymorphism:

Introduction

When an object is created from its class, the member variables and member functions are allocated memory spaces. The memory spaces have unique addresses. Pointer is a mechanism to access these memory locations using their address rather than the name assigned to them. You will study the implications and applications of this mechanism in detail in this chapter.

Pointer is a variable which can hold the address of a memory location rather than the value at the location. Consider the following statement

```
int num =84;
```

This statement instructs the compiler to reserve a 2-byte of memory location and puts the value 84 in that location. Assume that the compiler allocates memory location 1001 to num. Diagrammatically, the allocation can be shown as:

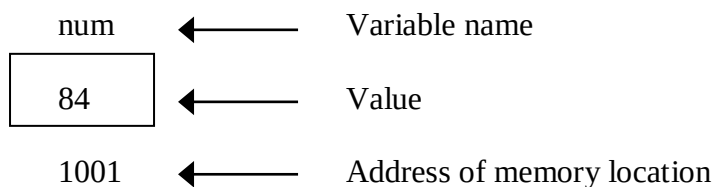


Figure 9.1

As the memory addresses are themselves numbers, they can be assigned to some other variable. For example, `ptr` be the variable to hold the address of variable `num`.

Thus, we can access the value of `num` by the variable `ptr`. We can say “`ptr` points to `num`” as shown in the figure below.

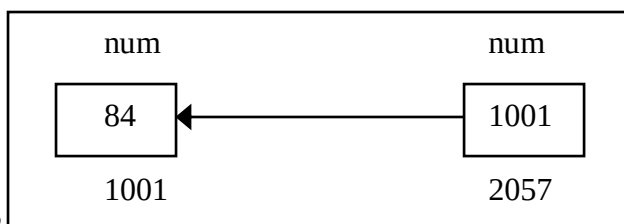


Fig 9.2

Pointers to Objects

An object of a class behaves identically as any other variable. Just as pointers can be defined in case of base C++ variables so also pointers can be defined for an object type. To create a pointer variable for the following class

```
class employee {
    int code;
    char name [20] ;
public:
    inline void getdata ( )= 0 ;
    inline void display ( )= 0 ;
};
```

The following codes is written

```
employee *abc;
```

This declaration creates a pointer variable abc that can point to any object of employee type.

this Pointer

C++ uses a unique keyword called "this" to represent an object that invokes a member function. 'this' is a pointer that points to the object for which this function was called. This unique pointer is called and it passes to the member function automatically. The pointer this acts as an implicit argument to all the member function, for e.g.

```
class ABC
{
    int a ;
    -----
    -----
};
```

The private variable 'a' can be used directly inside a member function, like

```
a=123;
```

We can also use the following statement to do the same job.

```
this → a = 123
```

e.g.

```
class stud
{
    int a;
public:
    void set (int a)
    {
        this → a = a; //here this point is used to assign a class level
    } 'a' with the argument 'a'
    void show ( )
    {
        cout << a;
    }
};
main ( )
{
    stud S1, S2;
```

```

        S1.bet (5);
        S2.show ( );
    }
    o/p = 5

```

Pointers to Derived Classes

Polymorphism is also accomplished using pointers in C++. It allows a pointer in a base class to point to either a base class object or to any derived class object. We can have the following Program segment show how we can assign a pointer to point to the object of the derived class.

```

class base
{
    //Data Members
    //Member Functions
};
class derived : public base
{
    //Data Members
    //Member functions
};

void main ( ) {
    base *ptr; //pointer to class base
    derived obj ;
    ptr = &obj ;    //indirect reference obj to the pointer
    //Other Program statements

}

```

The pointer ptr points to an object of the derived class obj. But, a pointer to a derived class object may not point to a base class object without explicit casting.

For example, the following assignment statements are not valid

```

void main ( )
{
    base obja;
    derived *ptr;
    ptr = &obja; //invalid....explicit casting required
    //Other Program statements
}

```

A derived class pointer cannot point to base class objects. But, it is possible by using explicit casting.

```

void main ( )
{
    base obj ;
    derived *ptr;    // pointer of the derived class
    ptr = (derived *) &obj;    //correct reference
    //Other Program statements
}

```

Student Activity

1. Define Pointers.
2. What are the various operators of pointer? Describe their usage.
3. How will you declare a pointer in C++?

LECTURE-31

Virtual Functions

Virtual functions, one of advanced features of OOP is one that does not really exist but it« appears real in some parts of a program. This section deals with the polymorphic features which are incorporated using the virtual functions.

The general syntax of the virtual function declaration is:

```
class use_detined_name{
private:
public:
virtual return_type function_name1 (arguments);
virtual return_type function_name2(arguments);
virtual return_type function_name3( arguments);
-----
};
```

To make a member function virtual, the keyword virtual is used in the methods while it is declared in the class definition but not in the member function definition. The keyword virtual precedes the return type of the function name. The compiler gets information from the keyword virtual that it is a virtual function and not a conventional function declaration.

For. example, the following declararion of the virtual function is valid.

```
class point {
intx;
inty;
public:
virtual int length ( );
virtual void display ( );
};
```

Remember that the keyword virtual should not be repeated in the definition if the definition occurs outside the class declaration. The use of a function specifier virtual in the function definition is invalid.

For example

```
class point {
intx ;
inty ;
public:
virtual void display ( );
};
virtual void point: : display ( ) //error
{
Function Body
}
```

A virtual function cannot be a static member since a virtual member is always a member of a particular object in a class rather than a member of the class as a whole.

```
class point {
int x ;
int y ;
public:
virtual static int length ( ); //error
```

```
};
    int point: : length ( )
    {
        Function body
    }
```

A virtual function cannot have a constructor member function but it can have the destructor member function.

```
class point {
int x ;
int y ;
public:
virtual point (int xx, int yy) ; // constructors, error
void display ( ) ;
int length ( ) ;
};
```

A destructor member function does not take any argument and no return type can be specified for it not even void.

```
class point {
int x ;
int y ;
public:
virtual point (int xx, int yy) ; //invalid
void display ( ) ;
int length ( ) ;
};
```

It is an error to redefine a virtual method with a change of return data type in the derived class with the same parameter types as those of a virtual method in the base class.

```
class base {
int x,y ;
public:
virtual int sum (int xx, int yy) ; //error
} ;
class derived: public base {
int z ;
public:
virtual float sum (int xx, int yy) ;
};
```

The above declarations of two virtual functions are invalid. Even though these functions take identical arguments note that the return data types are different.

```
virtual int sum (int xx, int yy) ; //base class
virtual float sum (int xx, int yy) ; //derived class
```

Both the above functions can be written with int data types in the base class as well as in the derived class as

```
virtual int sum (int xx, int yy) ; //base class
virtual int sum (int xx, int yy) ; //derived class
```

Only a member function of a class can be declared as virtual. A non member function (nonmethod) of a class cannot be declared virtual.

```
virtual void display ( ) //error, nonmember function
{
    Function body
}
```


Student Activity

1. What are virtual functions
2. What are pure virtual functions
3. Define Virtual destructors.

Late Binding

As we studied in the earlier unit, late binding means selecting functions during the execution. Though late binding requires some overhead it provides increased power and flexibility. The late binding is implemented through virtual functions as a result we have to declare an object of a class either as a pointer to a class or a reference to a class.

For example the following shows how a late binding or run time binding can be carried out with the help of a virtual function.

```
class base {
private :
int x;
float y;
public:
virtual void display ( ) ;
int sum ( ) ;
};
class derivedD : public baseA
{
private :
int x ;
float y;
public:
void display ( ) ; //virtual
int sum ( ) ;
};
void main ( )
{
    baseA *ptr ;
    derivedD objd ;
    ptr = &objd ;
    Other Program statements
    ptr->display ( ) ; //run time binding
    ptr->sum ( ) ; //compile time binding
}
```

Note that the keyword virtual is followed by the return type of a member function if a run time is to be bound. Otherwise, the compile time binding will be effected as usual. In the above program segment, only the display () function has been declared as virtual in the base class, whereas the sum () is nonvirtual. Even though the message is given from the pointer of the base class to the objects of the derived class, it will not

access the sum () function of the derived class as it has been declared as nonvirtual. The sum () function compiles only the static binding.

The following program demonstrates the run time binding of the member functions of a class. The same message is given to access the derived class member functions from the array of pointers. As function are declared as virtual, the C++ compiler invokes the dynamic binding.

```

#include <iostream.h>
#include <conio.h>
class baseA {
public :
virtual void display () {
cout<< "One \n";
}
};
class derivedB : public baseA
{
    public:
    virtual void display(){
        cout<< "Two\n"; }
};
class derivedC: public derivedB
{
    public:
    virtual void display ( ) {
        cout<< "Three \n"; }
};
void main ( ) {
    //define three objects
    baseA obja;
    derivedB objb;
    derivedC objc;
    base A *ptr [3]; //define an array of pointers to baseA
    ptr [0] = &obja;
    ptr [1] = &objb;
    ptr [2] = &objc;
    for ( int i = 0; i <=2; i ++ )
        ptr [i]->display ( ); //same message for all objects
    getch ( );
}

```

Output

One

Two

Three

The program listed below illustrates the static binding of the member functions of a class. In program there are two classes student and academic. The class academic is derived from class student. The two member function getdata and display are defined for both the classes. *obj is defined for class student, the address of which is stored in the object of the class academic. The functions getdata () and display () of student class are invoked by the pointer to the class.

```

#include<iostream.h>
#include<conio.h>
class student {
private:
int rollno;
char name [20];
public:
void getdata ( );
void display ( );

```

```

};
class academic: public student {
private:
char stream;
public:
void getdata ( );
void display ( ) ;
};
void student:: getdata ( )
{
    cout<< "enter rollno\n";
    cin>>rollno;
    cout<< "enter name \n";
    cin>>name;
}
void student:: display ( )
{
    cout<< "the student's roll number is "<<rollno<< "and name is"<<name ;
    cout<< endl;
}
void academic :: getdata ( )
{
    cout<< "enter stream of a student? \n";
    cin >>stream;
}
void academic :: display ( ) {
    cout<< "students stream \n";
    cout <<stream<< endl;
}
void main ( )
{
    student *ptr ;
    academic obj ;
    ptr=&obj;
    ptr->getdata ( ) ;
    ptr->display ( ) ;
    getche ( ) ;
}

```

output

```

enter rollno
25
enter name
raghu
the student's roll number is 25 and name is raghu

```

The program listed below illustrates the dynamic binding of member functions of a class. In this program there are two classes student and academic. The class academic is derived from student. Student function has two virtual functions getdata () and display (). The pointer for student class is defined and object . for academic class is created. The pointer is assigned the address of the object and function of derived class are invoked by pointer to student.

```

#include <iostream.h>
#include <conio.h>
class student {

```

```

private:
introllno;
char name [20];
public:
virtual void getdata ( );
virtual void display ( );
};
class academic: public student {
private :
char stream[10];
public:
void getdata { };
void display ( ) ;
};
void student: : getdata ( )
{
    cout<< “enter rollno\n”;
    cin >> rollno;
    cout<< “enter name \n”;
    cin >>name;
}
void student:: display ( )
{
    cout<< “the student’s roll number is”<<rollno<< “and name is”<<name;
    cout<< endl;
}
void academic: : getdata ( )
{
    cout << “enter stream of a student? \n”;
    cin>> stream;
}
void academic:: display ( )
{
    cout<< “students stream \n”;
    cout<< stream << endl;
}
void main ( )
{
    student *ptr ;
    academic obj ;
    ptr = &obj ;
    ptr->getdata ( );
    ptr->dlsplay ( );
    getch ( );
}

```

output
 enter stream of a student?
 Btech
 students stream
 Btech

LECTURE-32

Pure Virtual Functions

Generally a function is declared virtual inside a base class and we redefine it the derived classes. The function declared in the base class seldom performs any task.

The following program demonstrates how a pure virtual function is defined, declared and invoked from the object of a derived class through the pointer of the base class. In the example there are two classes employee and grade. The class employee is base class and the grade is derived class. The functions getdata () and display () are declared for both the classes. For the class employee the functions are defined with empty body or no code inside the function. The code is written for the grade class. The methods of the derived class are invoked by the pointer to the base class.

```
#include<iostream.h>
#include<conio.h>
class employee {
int code
char name [20] ;
public:
virtual void getdata ( ) ;
virtual void display ( ) ;
};
class grade: public employee
{
    char grd [90] ;
    float salary ;
public :
    void getdata ( ) ;
    void display ( );
};
void employee :: getdata ( )
{
}
void employee:: display ( )
{
}
void grade : : getdata ( )
{
    cout<< " enter employee's grade ";
    cin> > grd ;
    cout<< "\n enter the salary " ;
    cin>> salary;
}
void grade : : display ( )
{
    cout<<" Grade salary \n";
    cout<< grd<< " "<< salary<< endl;
```

```

    }
    void main ( )
    {
        employee *ptr ;
        grade obj ;
        ptr = &obj ;
        ptr->getdata ( ) ;
        ptr->display ( ) ;
        getche ( ) ;
    }

```

Output

```

enter employee's grade A
enter the salary 250000
Grade salary
A      250000

```

Object Slicing:

In C++, a derived class object can be assigned to a base class object, but the other way is not possible.

```

class Base { int x, y; };

class Derived : public Base { int z, w; };

int main()
{
    Derived d;
    Base b = d; // Object Slicing, z and w of d are sliced off
}

```

Object Slicing happens when a derived class object is assigned to a base class object, additional attributes of a derived class object are sliced off to form the base class object.

```

#include <iostream>
using namespace std;

class Base
{
protected:
    int i;
public:
    Base(int a)      { i = a; }
    virtual void display()
    { cout << "I am Base class object, i = " << i << endl; }
};

class Derived : public Base
{
    int j;
public:
    Derived(int a, int b) : Base(a) { j = b; }
    virtual void display()
    { cout << "I am Derived class object, i = "
      << i << ", j = " << j << endl; }
};

```

```
// Global method, Base class object is passed by value
void somefunc (Base obj)
{
    obj.display();
}

int main()
{
    Base b(33);
    Derived d(45, 54);
    somefunc(b);
    somefunc(d); // Object Slicing, the member j of d is sliced off
    return 0;
}
```

Output:

```
I am Base class object, i = 33
I am Base class object, i = 45
```

We can avoid above unexpected behavior with the use of pointers or references. Object slicing doesn't occur when pointers or references to objects are passed as function arguments since a pointer or reference of any type takes same amount of memory. For example, if we change the global method myfunc() in the above program to following, object slicing doesn't happen.

```
// rest of code is similar to above
void somefunc (Base &obj)
{
    obj.display();
}
// rest of code is similar to above
```

Output:

```
I am Base class object, i = 33
I am Derived class object, i = 45, j = 54
```

We get the same output if we use pointers and change the program to following.

```
// rest of code is similar to above
void somefunc (Base *objp)
{
    objp->display();
}
int main()
{
    Base *bp = new Base(33) ;
    Derived *dp = new Derived(45, 54);
    somefunc(bp);
    somefunc(dp); // No Object Slicing
    return 0;
}
```

Output:

```
I am Base class object, i = 33
I am Derived class object, i = 45, j = 54
```

Object slicing can be prevented by making the base class function pure virtual there by disallowing object creation. It is not possible to create the object of a class which contains a pure virtual method.

LECTURE-33

C++ Function Overriding

If base class and derived class have member functions with same name and arguments. If you create an object of derived class and write code to access that member function then, the member function in derived class is only invoked, i.e., the member function of derived class overrides the member function of base class. This feature in C++ programming is known as function overriding.

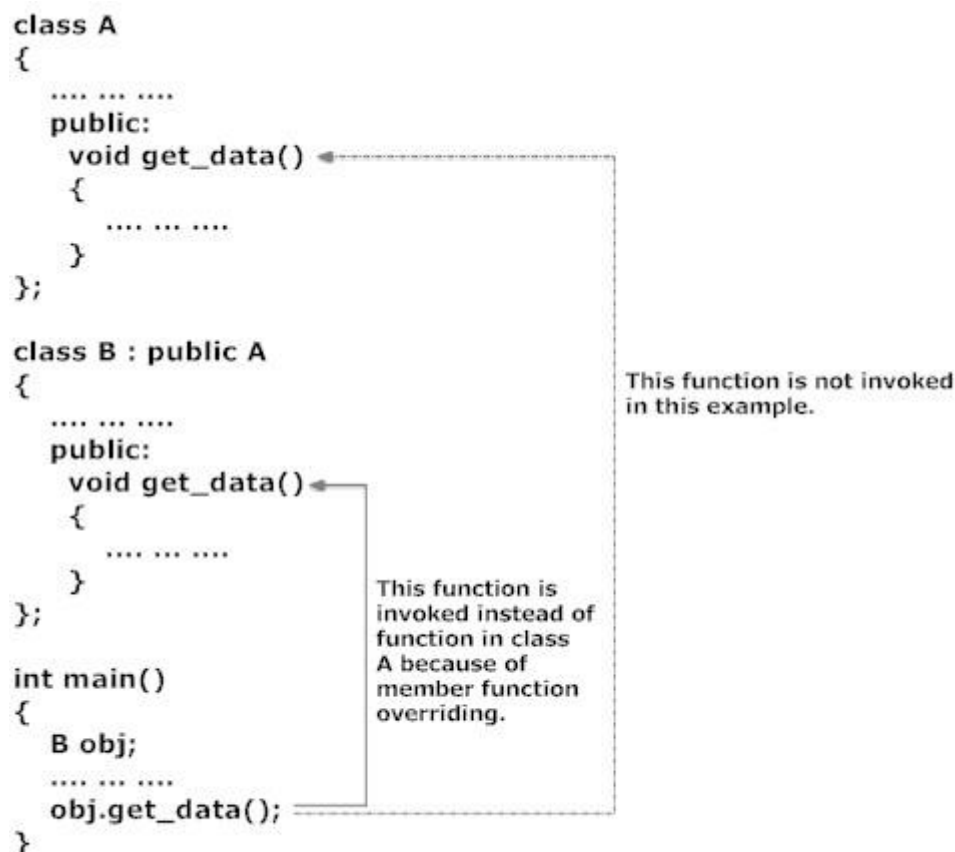


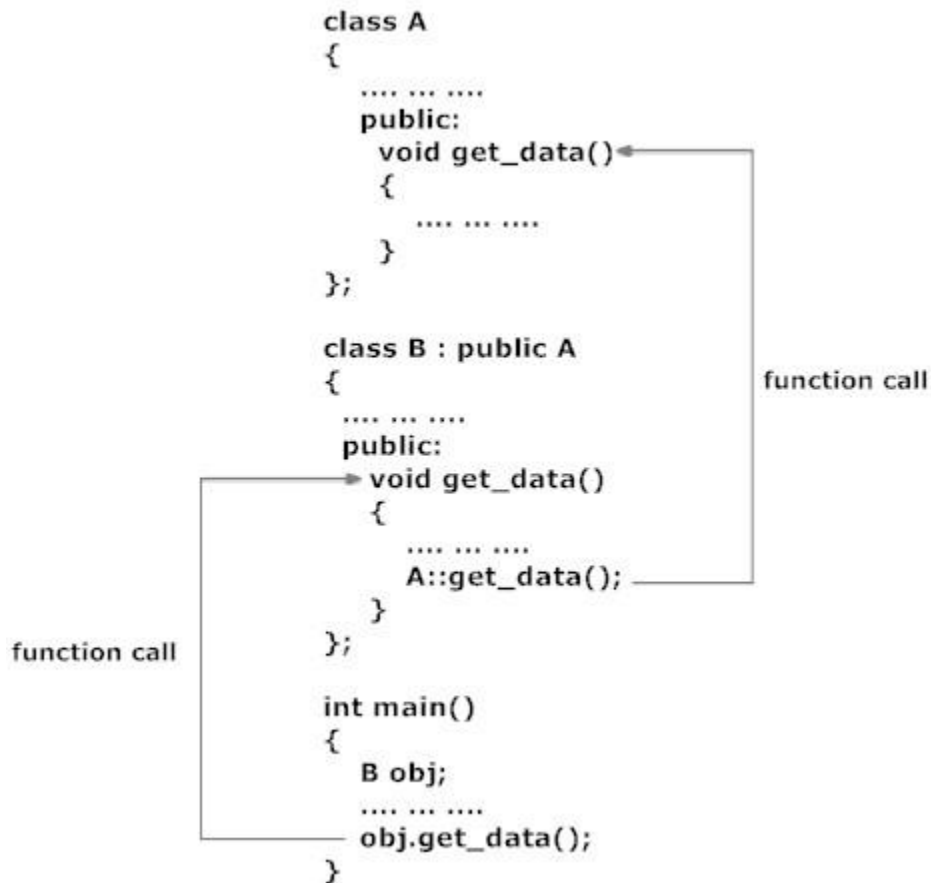
Figure: Member Function Overriding in C++

Accessing the Overridden Function in Base Class From Derived Class

To access the overridden function of base class from derived class, scope resolution operator `::`. For example: If you want to access `get_data()` function of base class from derived class in above example then, the following statement is used in derived class.

```
A::get_data; // Calling get_data() of class A.
```

It is because, if the name of class is not specified, the compiler thinks `get_data()` function is calling itself.



Abstract Class

Abstract Class is a class which contains atleast one Pure Virtual function in it. Abstract classes are used to provide an Interface for its sub classes. Classes inheriting an Abstract Class must provide definition to the pure virtual function, otherwise they will also become abstract class.

Characteristics of Abstract Class

1. Abstract class cannot be instantiated, but pointers and references of Abstract class type can be created.
2. Abstract class can have normal functions and variables along with a pure virtual function.
3. Abstract classes are mainly used for Upcasting, so that its derived classes can use its interface.
4. Classes inheriting an Abstract Class must implement all pure virtual functions, or else they will become Abstract too.

Pure Virtual Functions

Pure virtual Functions are virtual functions with no definition. They start with **virtual** keyword and ends with = 0. Here is the syntax for a pure virtual function,

```
virtual void f() = 0;
```

Example of Abstract Class

```
class Base    //Abstract base class
```

```

{
public:
virtual void show() = 0;      //Pure Virtual Function
};

class Derived:public Base
{
public:
void show()
{ cout << "Implementation of Virtual Function in Derived class"; }
};

int main()
{
Base obj;    //Compile Time Error
Base *b;
Derived d;
b = &d;
b->show();
}

```

Output : Implementation of Virtual Function in Derived class

In the above example Base class is abstract, with pure virtual **show()** function, hence we cannot create object of base class.

Why can't we create Object of Abstract Class ?

When we create a pure virtual function in Abstract class, we reserve a slot for a function in the VTABLE(studied in last topic), but doesn't put any address in that slot. Hence the VTABLE will be incomplete.

As the VTABLE for Abstract class is incomplete, hence the compiler will not let the creation of object for such class and will display an error message whenever you try to do so.

LECTURE-34

Exception Handling:

Exception refers to unexpected condition in a program. The unusual conditions could be faults, causing an error which in turn causes the program to fail. The error handling mechanism of c++ is generally referred to as exception handling.

Generally , exceptions are classified into synchronous and asynchronous exceptions.. The exceptions which occur during the program execution, due to some fault in the input data or technique that is not suitable to handle the current class of data. with in a program is known as synchronous exception.

Example:

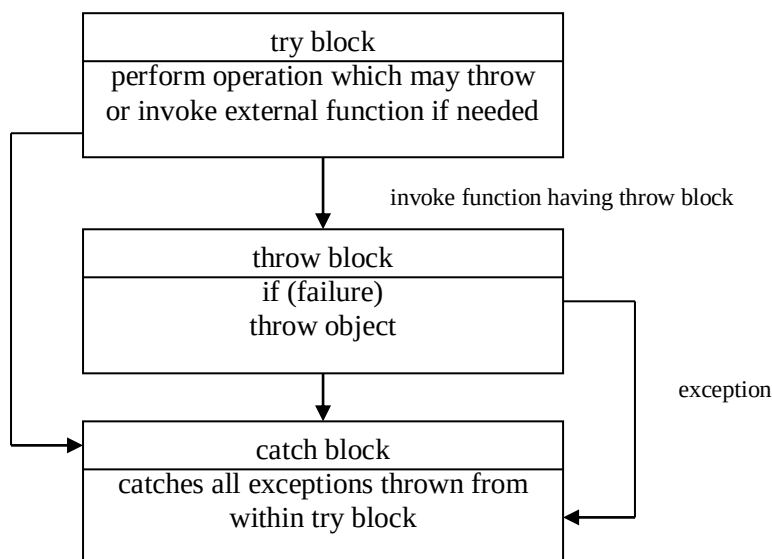
errors such as out of range,overflow,underflow and so on.

The exceptions caused by events or faults unrelated to the program and beyond the control of program are asynchronous exceptions.

For example, errors such as keyboard interrupts, hardware malfunctions, disk failure and so on.

exception handling model:

When a program encounters an abnormal situation for which it is not designed, the user may transfer control to some other part of the program that is designed to deal with the problem. This is done by throwing an exception. The exception handling mechanism uses three blocks: try, throw and catch. The try block must be followed immediately by a handler, which is a catch block. If an exception is thrown in the try block the program control is transferred to the appropriate exception handler. The program should attempt to catch any exception that is thrown by any function. The relationship of these three exceptions handling constructs called the exception handling model is shown in figure:



throw construct:

The keyword throw is used to raise an exception when an error is generated in the computation. the throw expression initialize a temporary object of the type T used in throw (T arg).

syntax:

```
throw T;
```

catch construct:

The exception handler is indicated by the catch keyword. It must be used immediately after the statements marked by the try keyword. The catch handler can also occur immediately after another catch. Each handler will only evaluate an exception that matches.

syn:

```
catch(T)
{
// error messages
}
```

try construct:

The try keyword defines a boundary within which an exception can occur. A block of code in which an exception can occur must be prefixed by the keyword try. Following the try keyword is a block of code enclosed by braces. This indicates that the prepared to test for the existence of exceptions. If an exception occurs, the program flow is interrupted.

```
try
{
...
if (failure)
    throw T;
}
catch(T)
{
...
}
```

example:

```
#include<iostream.h>
void main()
{
int a,b;
cout<<"enter two numbers:";
cin>>a>>b;
try
{
if (b==0)
    throw b;
else
    cout<<a/b;
}
catch(int x)
{
cout<<"2nd operand can't be 0";
}
}
```

LECTURE-35

Array reference out of bound:

```
#define max 5
class array
{
private:
    int a[max];
public:
    int &operator[](int i)
    {
        if (i<0 || i>=max)
            throw i;

        else
            return a[i];
    }
};

void main()
{
    array x;
    try
    {
        cout<<"trying to refer a[1]..."
        x[1]=3;
        cout<<"trying to refer a[13]..."
        x[13]=5;
    }
    catch(int i)
    {
        cout<<"out of range in array references...";
    }
}
```

multiple catches in a program

```
void test(int x)
{
    try{
        if (x==1)
            throw x;
        else if (x==-1)
            throw 3.4;
        else if (x==0)
            throw 's';
    }
    catch (int i)
    {
        cout<<"caught an integer...";
    }
    catch (float s)
    {
        cout<<"caught a float...";
    }
}
```

```

}
catch (char c)
{
cout<<"caught a character...";
}}
void main()
{
test(1);
test(-1);
test(0);
}

```

catch all

```

void test(int x)
{
try{
if (x==1)
    throw x;
else if (x==-1)
    throw 3.4;
else if (x==0)
    throw 's';
}
catch (...)
{
cout<<"caught an error...";
}
}

```